

Steer Phantom Liquidity Hook Audit Summary

Prepared by Cyfrin

Version 2.0

Head of Security Research

Dacian (Proprietary AI)

Lead Auditors

0x3b33

ret2basic

deth

November 3, 2026

Contents

1 About Cyfrin 2

2 About This Document 2

3 Disclaimer 2

4 Copyright and License 2

5 Audit Overview 3

6 Audit Scope 3

7 Risk Classification 4

8 Protocol Summary 4

9 Attestation 4

1 About Cyfrin

Cyfrin is a Web3 security company dedicated to bringing industry-leading protection and education to our partners and their projects. Our goal is to create a safe, reliable, and transparent environment for everyone in Web3 and DeFi. Learn more about us at cyfrin.io.

2 About This Document

This document is a public summary of the security audit Cyfrin performed for Steer on Phantom Liquidity Hook. It records the repositories and commits reviewed, the timeline, auditors, in-scope files, and the number and final status of findings by severity. It closes with Cyfrin's attestation of the review's scope and final remediation status.

Technical details of individual findings are omitted from this public summary to limit disclosure of security-sensitive information. The complete audit report was delivered privately to Steer, who retain it as the full record of the engagement. Recipients of the full report can use the SHA-256 hash in this summary's attestation to confirm that their copy matches the referenced report.

3 Disclaimer

The Cyfrin team makes every effort to find as many vulnerabilities in the code as possible in the given time but holds no responsibility for the findings in this document. A security audit by the team does not endorse the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the in-scope code being audited. Unless the scope explicitly notes otherwise, the review is limited to the application layer and does not extend to the correctness of API/SDK/execution environment implementations, compilers, standard and third-party libraries, underlying operating systems and hardware.

4 Copyright and License

© 2026 Cyfrin Inc. All rights reserved. Material from Steer or third parties remains the property of its owners.

This summary may be shared only in complete and unmodified form, so that any copy can be checked against the hash in its attestation. Short quotations with attribution to Cyfrin are permitted, provided they do not misrepresent this summary's scope or results. Except as permitted by applicable law, no part of this summary may otherwise be copied, modified, adapted, sold or sublicensed, or used to create derivative works, without Cyfrin's prior written permission.

Cyfrin and the Cyfrin logo are trademarks of Cyfrin Inc. and may not be used to suggest that Cyfrin has reviewed or endorses anything outside this summary's scope, or to misrepresent its results.

5 Audit Overview

Over the course of 20 days, the Cyfrin team conducted an audit on the Phantom Liquidity Hook code provided by Steer. The findings consist of 5 Medium & 41 Low severity issues with the remainder being informational and gas optimizations.

Summary

Project Name	Phantom Liquidity Hook
Repository	phantom-liquidity-hook
Commit	2fa1c8a25e12...
Fix Commit	818f207a2e01...
Audit Timeline	Sep 14th - Oct 9th, 2026
Methods	Manual Review, Proprietary AI

Issues Found

Severity	Found	Resolved	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	5	5	0
Low Risk	41	32	9
Informational	19	11	8
Gas Optimizations	5	1	4
Total	70	49	21

How to read the statuses

Resolved means Cyfrin verified the remediation documented in the full report with a Cyfrin verification note. Acknowledged means Steer chose to retain the reported behavior or accept the residual risk; this does not constitute resolution or endorsement by Cyfrin. Finding-specific explanations and verification notes are recorded in the full report.

6 Audit Scope

The audit scope was limited to:

```
src/HybridCollateralSettlementHook.sol
src/SignedCollateralSwapRouter.sol
src/IssuerSettlementSource.sol
src/CollateralSettlementController.sol
src/RegisteredIssuerBrokerAdapter.sol
src/LifiHookExecutor.sol
src/risk/RiskConfig.sol
src/risk/RollingCapManager.sol
src/libraries/SettlementPolicy.sol
src/libraries/SignedQuoteHash.sol
src/libraries/SignedSwapQuote.sol
```

```
src/libraries/LiquidationHookData.sol
src/oracle/GeometricMeanOracleAdapter.sol
script/production/DeployRegisteredIssuerStack.s.sol
```

7 Risk Classification

	Impact: High	**Impact: Medium**	**Impact: Low**
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

8 Protocol Summary

Stablecoin trading and issuance run through Uniswap v4: a pool hook intercepts swaps on managed collateral and stablecoin pools and routes the swap legs to external settlement that mints or redeems the stablecoin through a registered issuer, with off-market pricing bounded by EIP-712 signed quotes and oracle checks. The stack deploys as a single unit with one admin; user tokens sit with the Uniswap v4 pool manager outside settlement, and mint and redeem value moves through the hook's privileged settlement path rather than a public minting function.

- `HybridCollateralSettlementHook` is the Uniswap v4 hook on the protocol's pools: it validates pool parameters, lets only its owner initialize a pool, enforces global safe modes, oracle deviation gating and per-token amount and settlement-delta caps, can enforce an oracle price floor and per-block caps when configured, and routes swap legs to external settlement sources. The registered-issuer deploy script requires the per-token caps and leaves the per-block cap and the minimum oracle price disabled.
- `IssuerSettlementSource` and `CollateralSettlementController` form the settlement layer for registered-issuer markets, validating EIP-712 signed quotes and routing mint and redeem legs through the broker adapter; the admin-installed settlement hook is the only caller permitted to drive the settlement entry points.
- `RegisteredIssuerBrokerAdapter` executes those legs against the external registered issuer through AtomicBroker's USDM1 permit rail, using signed price attestations.
- `SignedCollateralSwapRouter` is the trader-facing router for deadline-bound signed-quote swaps; `LifiHookExecutor` adapts the same hook path to fill same-chain LI.FI escrow orders under an operator role.
- `RiskConfig` and `RollingCapManager` hold per-collateral and global risk parameters, mint and redeem caps, and emergency safe modes. `GeometricMeanOracleAdapter` prices collateral from multiple Chainlink feeds by geometric mean and grades its oracle mode by the spread between the valid feeds.

9 Attestation

Cyfrin attests that it performed a security review of the Phantom Liquidity Hook code provided by Steer.

The repositories, commits, review period and findings with their final status are in the Summary and Issues Found tables in Section 5, and the in-scope files in Section 6. This attestation is limited to the documented scope, audited commits and mitigations specifically reviewed by Cyfrin.

Cyfrin publishes this document and its SHA-256 hash at https://github.com/Cyfrin/cyfrin-audit-reports. To check a copy on macOS or Linux, run the following in the folder that contains it; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name. Before running it, check that the URL is Cyfrin's repository.

```
U=https://github.com/Cyfrin/cyfrin-audit-reports/raw/main/reports/sha256
N=2026-11-03-cyfrin-steer-phantom-liquidity-hook-v2.0.pdf
F=$N
H=$(curl -fsSL $U/$N.sha256 | cut -d' ' -f1)
echo "$H *$F" | shasum -a 256 -c
```

Cyfrin's full audit report with all findings provided to Steer has the SHA-256 hash and file name below. To check a copy of it, first check this document as above, then run the following in the folder that contains the full report; it prints `OK` when the copy matches. If the copy was renamed, set F to its file name.

```
H=d0b870e9de374e678e68c2e5367b7c4af73e19a3f596cf7a5f451f160aebdb94
F=2026-11-03-cyfrin-steer-phantom-liquidity-hook-v2.0-full.pdf
echo "$H *$F" | shasum -a 256 -c
```

On Linux, `sha256sum -c` can be used in place of `shasum -a 256 -c`. On Windows, run `Get-FileHash <file> -Algorithm SHA256` in PowerShell and compare the result with the hash.